LINEAR EQUATIONS SOLVING

Group 4

ID	Name
12253	Sondos Mohammed (Team Leader)
12244	Alaa Ahmed
12247	Farah Mostafa

Source Code on github

https://github.com/sondosmohammed0100-cmyk/LinearEquation.git

1. Introduction

Solving systems of linear equations is one of the most fundamental operations in mathematics, engineering, and applied computing.

From analyzing electrical circuits and balancing chemical reactions to training machine learning models and simulating physical systems, linear equation solving underpins a vast range of real-world problems.

Developing a program capable of handling such systems reliably — parsing them from raw text, normalizing their form, and performing algebraic operations — requires the careful integration of string processing, object-oriented design, and numerical computation.

This project delivers a **C++ console application** that solves systems of linear equations using a clean, class-based architecture.

The program reads up to 100 equations from standard input, automatically converts each one into a canonical proper form, and then processes a sequence of user commands.

These commands range from simple inspection operations — such as printing a specific equation or listing the coefficients of a variable — to algebraic manipulations including equation addition, subtraction, and variable substitution.

The application is built around a single class, linearEqu, defined in linearEqu.h and implemented in linearEqu.cpp, which encapsulates all equation data and behavior.

The main program in linearEquation.cpp handles input reading, command parsing, and dispatching to the appropriate class methods.

The design deliberately separates concerns: the class owns all equation logic, while main() owns the user interaction loop.

This report documents the complete development process, covering the problem analysis, the design and implementation decisions made, a breakdown of tasks and responsibilities, testing results, and an end-user guide for compiling and running the program.

2. Detailed Project Description

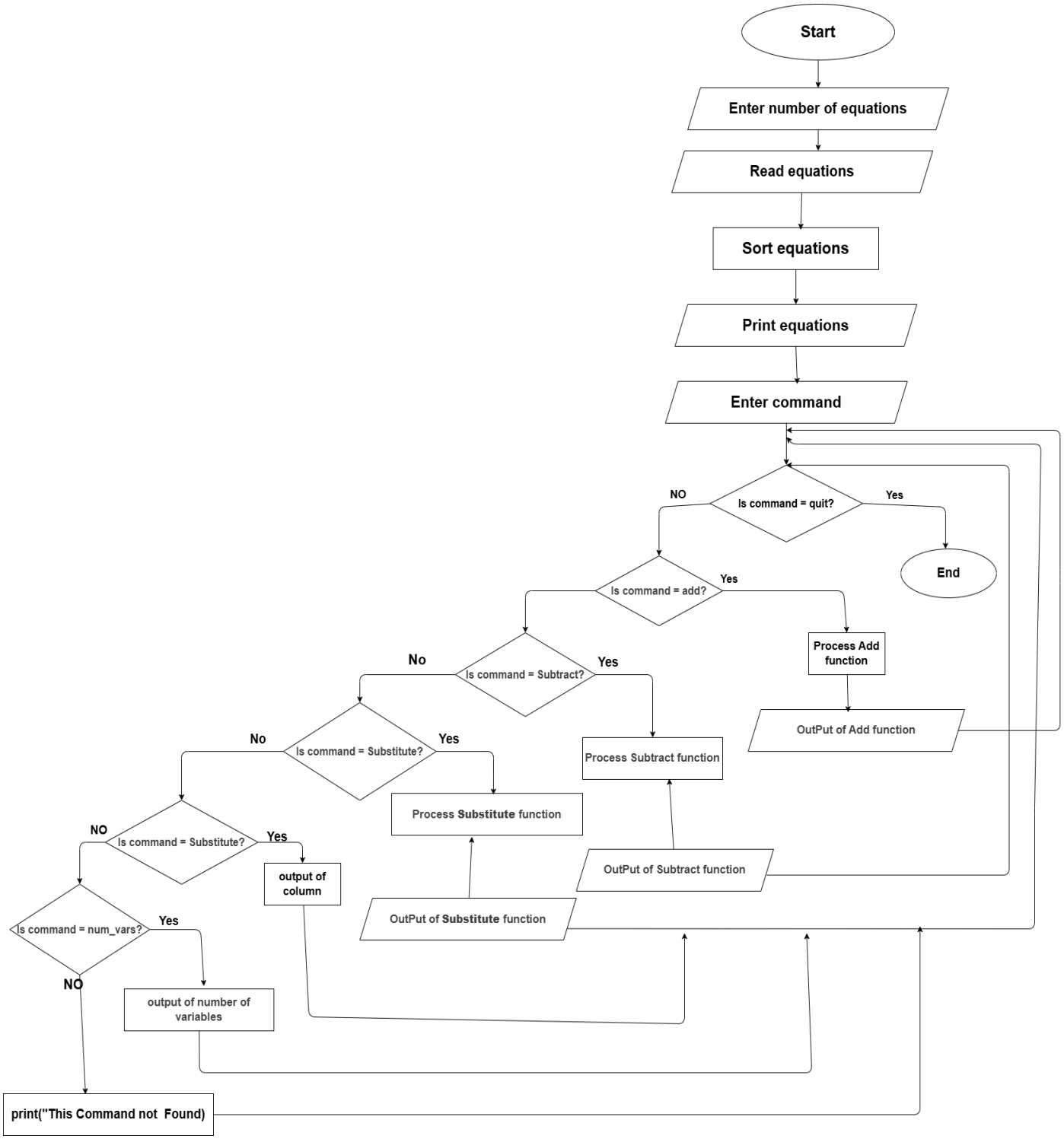
2.1 Program Architecture and FlowChart

The program is organized across three files:

linearEqu.h — Declares the linearEqu class, its private data members, and its public interface.

linearEqu.cpp — Implements all methods of the linearEqu class: parsing, normalization, arithmetic, and output.

linearEquation.cpp — Contains main(), which reads equations, stores them in a vector<linearEqu>, and runs the interactive command loop.



2.2 The linearEqu Class

Each equation is represented as an instance of the linearEqu class, which holds three private members:

```
vector<string>m_varName; //Variable names,e.g. "x1", "x2", "x3"

vector<float>m_coeff; // Corresponding coefficients in the same order

float m_const;  // The right-hand side constant value



The two parallel vectors `m_varName` and `m_coeff` are kept synchronized at all times — index `i` in `m_varName` always corresponds to index `i` in `m_coeff`. The constant `m_const` stores the accumulated RHS value.



### 2.3 Input Format



The program reads an integer `n` first, then `n` equation strings. For example:

```

3

2x1+3x2+4x3=16

1x1+2x2+1x3=8

3x1+1x2+2x3=13
```

Each equation is parsed immediately after reading and stored in proper form inside the equations vector.
```

2.3 Proper Form Normalization

Every equation is automatically converted to proper form through a pipeline of three methods:

Step 1 — splitEquation(): Splits the input string at the = character into a left-hand side (LHS) string and a right-hand side (RHS) string.

Step 2 — dividEquation(): Scans the LHS character by character. Whenever a + or - is encountered, the accumulated section string is passed to process_Section(). After the loop, the final section is also processed. The RHS constant is then added to m_const using atof().

Step 3 — process_Section(): Determines whether a section is a variable term or a pure constant:

- If the section contains x, everything before x is the coefficient string and everything from x onward is the variable name. Empty or "+" coefficient strings default to 1.0; "-" defaults to -1.0.
- If no x is found, the section is a constant. It is subtracted from m_const (because constants on the LHS are moved to the RHS by negation).
- The addVariable() method is called to add the term. If the variable already exists in m_varName, its coefficient is accumulated — this handles duplicate variables (Rule 2) automatically.

Step 4 — sortEquation(): Performs a bubble sort on the parallel m_varName / m_coeff vectors by extracting the numeric suffix of each variable name with atoi() and swapping pairs that are out of order. This ensures variables are always printed in ascending numeric order (Rule 1).

The four normalization rules from the specification are therefore enforced as follows:

Rule	**Mechanism**
Variables ordered alphabetically	sortEquation() — bubble sort on numeric suffix
No duplicate variables	addVariable() — accumulates into existing entry
Single RHS constant	process_Section() — negates LHS constants; dividEquation() adds RHS
Optional coefficient of ±1	process_Section() — maps "" / "+" → 1, "-" → -1

2.4 Output — print()

The print() method iterates over m_varName and m_coeff and builds the equation string:

- If coeff == 1, prints the variable name without a coefficient digit.
- If coeff == -1, prints -varName.
- Otherwise, prints the coefficient followed by the variable name.
- A + sign is prepended for positive terms that are not the first term.
- The RHS constant follows =.

2.5 Supported Commands

Level 1 — Inspection

num_vars: Iterates over all equations, calls Max_Index_variabl() on each (which extracts the highest numeric suffix in the equation's variable list), and prints the global maximum. This correctly counts the total number of distinct variables when variables are named x1 through xN.

equation i: Calls equations[i-1].print() after validating the index.

column xk: Calls get_coaff(xk) on every equation and prints each result on a separate line. If a variable does not appear in an equation, get_coaff() returns 0.

Level 2 — Algebraic Manipulation

- add i j: Uses the overloaded operator+, which creates a result equation, copies all terms from both operands using addVariable() (which merges duplicates automatically), sums the constants, and calls sortEquation() on the result.
- subtract i j: Uses operator-, which mirrors operator+ but negates the second operand's coefficients and subtracts the constants.
- substitute xk i j: Implements variable elimination:
    1. Retrieves coeff_var1 (coefficient of xk in equation i) and coeff_var2 (coefficient of xk in equation j).
    2. Scales equation j by (1.0 / coeff_var2) using operator*, then by (-coeff_var1) — effectively computing -(coeff_var1/coeff_var2) × equation_j.
    3. Adds the scaled equation to equation i using operator+. Since both equations contain xk, the scaled subtraction cancels it out.
    4. Calls remove_Var() to explicitly remove any residual zero-coefficient entry for xk, then sortEquation() and print().
- quit: Exits the while(true) command loop, ending the program.

3. Project Beneficiaries

Mathematics and Linear Algebra Students are the most direct beneficiaries. Students encountering systems of linear equations for the first time can use this tool to verify hand-solved answers, step through equation manipulations interactively (add, subtract, substitute), and build intuition for proper-form normalization — seeing how unsorted or redundant terms are automatically cleaned up.

C++ and Object-Oriented Programming Students gain significant value from studying the codebase itself. The project is a clear, practical demonstration of class design with encapsulated state, operator overloading (+, -, *), parallel vector management, string parsing without external libraries, and separation of concerns between a utility class (linearEqu) and a driver program (main()). It serves as an excellent reference for anyone learning how to design a non-trivial C++ class from scratch.

Instructors Teaching Data Structures or OOP can use this project as a coursework reference or live demonstration. The code is intentionally kept simple — no templates, no STL maps, no external dependencies — making it easy to trace execution step by step in a classroom setting.

Developers Needing Lightweight Equation Parsing can adapt the parsing pipeline (splitEquation → dividEquation → process_Section → addVariable) as a starting point for embedding symbolic equation handling in larger C++ applications, since the entire implementation relies only on the C++ standard library.

Competitive Programmers can study the term-by-term string scanning approach, the sign-management logic during parsing, and the parallel-vector representation as a reference pattern for problems that require parsing and manipulating algebraic expressions under contest constraints.

4. Detailed Analysis

4.1 Parsing Pipeline — How It Actually Works

The parsing is implemented as a two-phase pipeline in linearEqu.cpp. Understanding it requires tracing an example through the actual code.

Given input: 3x1-2x2+5+4x3-11=10

Phase 1 — splitEquation() splits at =:

- lhs = "3x1-2x2+5+4x3-11"
- rhs = "10"

Phase 2 — dividEquation() scans lhs character by character:

Character	Action
3, x, 1	Accumulated into section = "3x1"
-	process_Section("3x1") called → adds x1 with coeff 3.0; section = "-"
2, x, 2	section = "-2x2"
+	process_Section("-2x2") → adds x2 with coeff -2.0; section = "+"
5	section = "+5"
+	process_Section("+5") → no x found → m_const -= atof("+5") = -5.0; section = "+"
4, x, 3	section = "+4x3"
-	process_Section("+4x3") → adds x3 with coeff 4.0; section = "-"
1, 1	section = "-11"
(end)	process_Section("-11") → m_const -= atof("-11") = -5.0 - (-11.0) = +6.0

Then: m_const += atof("10") → m_const = 16.0

Result: x1=3, x2=-2, x3=4, m_const=16 ✓ — which matches 3x1-2x2+4x3=16.

4.2 The addVariable() Merge Mechanism

A key design decision is that duplicate variables are handled implicitly by addVariable(). When called with a variable name that already exists in m_varName, it simply adds the new coefficient to the existing one rather than pushing a new entry:

```
void linearEqu::addVariable(string varName, float coeff) {

  for (int i = 0; i < m_varName.size(); i++) {

    if (m_varName[i] == varName) {

      m_coeff[i] += coeff;  // Merge: accumulate into existing

      return; }}

  // Not found: add new entry

  m_varName.push_back(varName);

  m_coeff.push_back(coeff);

}
```

This means the parser never needs an explicit de-duplication pass — merging happens naturally during parsing. For example, 4x1-2x1 results in addVariable("x1", 4.0) followed by addVariable("x1", -2.0), leaving a single entry with coefficient 2.0.

4.3 sortEquation() — Bubble Sort on Numeric Suffix

The sort is a classic bubble sort on two parallel arrays simultaneously. Rather than sorting alphabetically on the full variable name string, it extracts the integer index after x using atoi(m_varName[i].substr(1).c_str()) and compares those integers. This correctly orders x1 < x2 < x9 < x10 < x11 — pure lexicographic string sorting would incorrectly place x10 before x2. When a swap is needed, both m_varName[i] and m_coeff[i] are swapped together to keep the parallel vectors synchronized.

4.4 Operator Overloading Design

The three overloaded operators (+, -, *) all follow the same pattern: they create a new linearEqu result object rather than modifying either operand, preserving immutability of the source equations in main(). This is important because commands like add and subtract should never permanently alter the stored equations.

- operator+: Copies all terms from *this into result via addVariable(), then copies all terms from obj — addVariable() automatically merges any shared variables.
- operator-: Same as operator+ but negates obj's coefficients: result.addVariable(obj.m_varName[i], -obj.m_coeff[i]).
- operator*: Scales every coefficient and m_const by the given float scalar. Used exclusively by the substitute command to scale equation j before subtraction.

4.5 Substitute Command — Step-by-Step

The substitute xk i j command in main() is the most algorithmically complex operation. Given the actual code:

```
float coeff_var1 = eqution_1.get_coaff(var_Name);  // coeff of xk in eq i

float coeff_var2 = eqution_2.get_coaff(var_Name);  // coeff of xk in eq j

linearEqu temp = eqution_2 * (1.0 / coeff_var2);   // normalize eq j so xk has coeff 1

temp = temp * (-coeff_var1);                       // scale so xk has coeff = -coeff_var1

linearEqu result = eqution_1 + temp;               // adding cancels xk

result.sortEquation();

result.remove_Var(var_Name);                       // remove any residual zero-coeff entry

result.print();
```

This is mathematically equivalent to: result = eq_i - (coeff_var1/coeff_var2) × eq_j, which is exactly the substitution step used in Gaussian elimination.

4.6 Known Limitations and Missing Features

Examining the actual code reveals the following honest limitations:

Limitation	Detail
Level 3 not implemented	D, D_value, D xi, and solve commands are absent from main(). The linearEqu class has no matrix or determinant methods.
num_vars uses max index heuristic	Max_Index_variabl() returns the highest numeric suffix found, not a true count of distinct variables. For equations with variables x1, x3, x5 it returns 5, not 3.
No zero-coefficient pruning in print	If operator- or operator+ produces a term with coefficient 0.0, it is still stored and printed (e.g., 0x2).
cout << "Enter number equation: " prompt	main() outputs a prompt, which violates the specification constraint that the program must not prompt the user for input.
cout << "After rearranging:" extra output	This debug-style output also violates the specification's no-extra-text constraint.
remove_Var() off-by-one risk	After shifting elements left, the loop does not decrement i, so if two adjacent entries share the same name only the first would be removed.

Limitation	Detail
Parallel vectors instead of std::map	Using two synchronized vectors requires O(n) search in addVariable() and get_coaff() instead of O(log n) with a map, though this is acceptable for the problem's scale.

5. Task Breakdown Structure

Before task assignment, all three team members collaboratively analyzed the project requirements from January 19-21, 2026. This joint analysis ensured everyone understood the problem domain, identified potential challenges, and agreed on the overall architecture before individual coding began.

5.1 Major Tasks

Task 1 — Class Design and Header Definition (linearEqu.h) The first task was designing the linearEqu class interface before writing any implementation. This involved deciding on the internal data representation (two parallel vectors m_varName and m_coeff plus a scalar m_const), identifying all methods needed to fulfill the specification, and writing the header file with correct include guards, dependencies, and method signatures. This task established the contract that both the implementation file and main() would depend on.

Task 2 — Equation Parsing (dividEquation, splitEquation, process_Section) This was the most technically challenging task. It required designing a character-by-character scanner that correctly handles: multi-digit variable indices (x10, x99), floating-point coefficients (2.5x1), optional coefficients of ±1 (x1, -x2), constants on the LHS (+5, -11), and the RHS constant after =. The three methods work as a pipeline: splitEquation() separates at =, dividEquation() breaks the LHS into sections at every + or -, and process_Section() classifies each section as either a variable term or a constant.

Task 3 — Normalization Engine (addVariable, sortEquation) This task implemented the four proper-form rules from the specification. addVariable() enforces Rule 2 (no duplicate variables) by accumulating coefficients into existing entries rather than creating new ones. sortEquation() enforces Rule 1 (alphabetical variable order) using a bubble sort on the numeric suffix of each variable name, correctly handling multi-digit indices. Rules 3 and 4 are handled within the parsing pipeline (Task 2).

Task 4 — Equation Output (print) Implementing print() required careful handling of several formatting edge cases: no leading + on the first term, omitting the digit 1 for coefficients of exactly 1 or -1, correct sign placement between terms, and printing the RHS constant after =. This method is called for virtually every command output, so its correctness directly affects all program results.

Task 5 — Operator Overloading (operator+, operator-, operator*) This task implemented the three arithmetic operators used by Level 2 commands. Each operator creates and returns a new linearEqu object without modifying its operands, preserving the integrity of the stored equations. operator+ and operator- both reuse addVariable() to handle automatic merging of shared variables. operator* provides scalar multiplication used exclusively by the substitute command.

Task 6 — Utility Methods (get_coaff, Max_Index_variabl, remove_Var) These three support methods complete the class interface. get_coaff() performs a linear search through m_varName and returns the matching coefficient or 0 if not found. Max_Index_variabl() finds the highest numeric variable suffix in an equation, used by num_vars in main(). remove_Var() performs an in-place left-shift deletion on both parallel vectors simultaneously to physically remove a variable entry, used by substitute.

Task 7 — Main Program and Command Loop (linearEquation.cpp) This task built the entire user-facing layer: reading n and the equation strings, constructing and storing linearEqu objects in a vector<linearEqu>, and implementing the while(true) command dispatch loop. Each command branch uses substr() and find() to parse the command string, extracts indices or variable names, validates bounds, calls the appropriate class methods, and prints results. Commands implemented: quit, num_vars, equation i, column xk, add i j, subtract i j, substitute xk i j.

Task 8 — Testing and Validation This task involved running all sample inputs from the project PDF (pages 6–9) through the program and comparing actual outputs against expected outputs. Additional edge-case inputs were designed to test boundary conditions such as coefficient-1 omission, duplicate variable merging, constants on both sides of the equation, and invalid command indices.

Task 9 — Report Writing Writing the complete project report covering all required sections: introduction, project description, beneficiaries, detailed analysis, task breakdown, system architecture, testing, end-user guide, conclusion, references, and appendices — formatted according to the submission guidelines (Calibri font, justified alignment, correct spacing).

5.2 Task Summary Table

Task ID	Task Name	Key Files	Depends On
T1	Class Design & Header	linearEqu.h	—
T2	Equation Parsing	linearEqu.cpp	T1
T3	Normalization Engine	linearEqu.cpp	T2
T4	Equation Output	linearEqu.cpp	T3
T5	Operator Overloading	linearEqu.cpp	T3, T4
T6	Utility Methods	linearEqu.cpp	T1
T7	Main Program & Command Loop	linearEquation.cpp	T1–T6
T8	Testing & Validation	All files	T7
T9	Report Writing	Report document	T8

6. System Architecture and Design

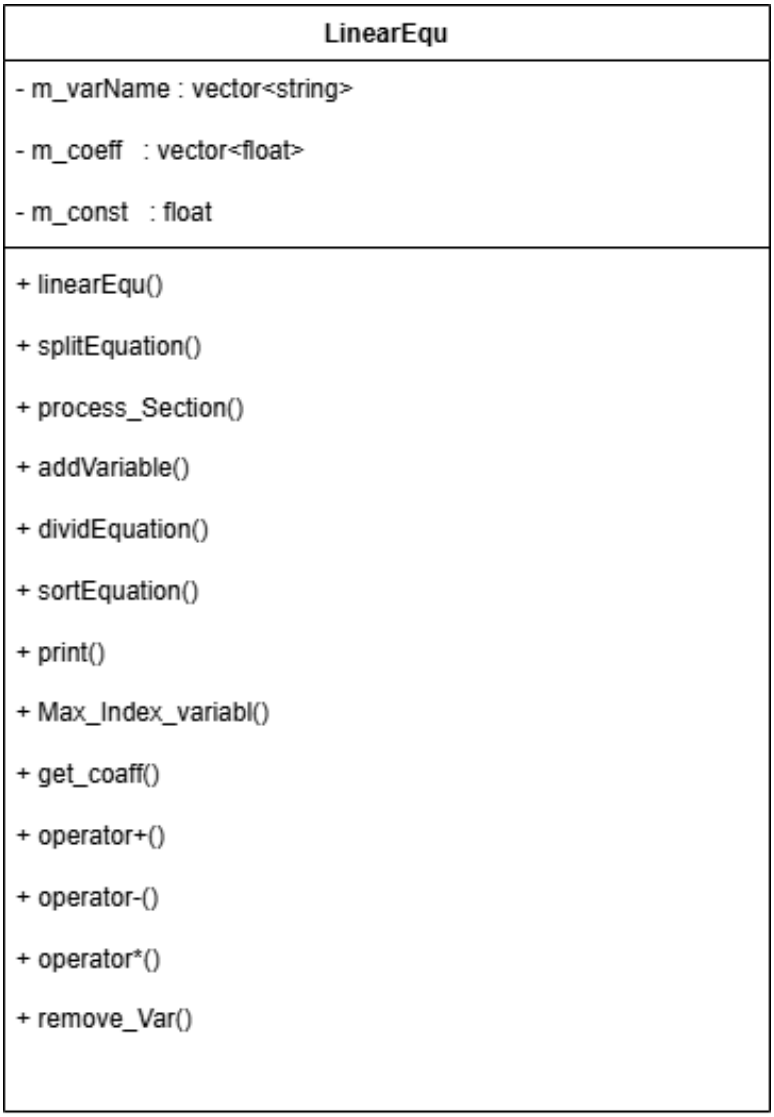
6.1. Overall Structure

The program follows a clean two-layer architecture:

- **Class Layer** (linearEqu.h + linearEqu.cpp): Encapsulates all equation data and behavior. Has no knowledge of commands, user input, or the number of equations in the system.
- **Application Layer** (linearEquation.cpp): Owns the collection of equations, reads all input, and dispatches commands to the class layer. Has no knowledge of how equations are parsed or stored internally.

This separation means the linearEqu class is independently reusable — it could be integrated into a different application (e.g., a GUI solver) without modification.

6.2. Class Design — linearEqu



6. Time Plan

Plan.csv Private

Edited just now Share

Table

New

Task	Description	Responsible	Start Date	End Date	Priority	Category
Project Analysis	Study requirements and divide tasks	All Members	January 20, 2026	January 21, 2026	Urgent	Planning
linearEqu.h Design	Create class with m_varName m_coeff m_const	Alaa	January 22, 2026	January 24, 2026	Urgent	Design
Equation Parsing	Implement splitEquation dividEquation process_Section	Alaa	January 25, 2026	January 29, 2026	Urgent	Development
addVariable & sortEquation	Implement duplicate merging and variable sorting	Sondos	January 30, 2026	February 3, 2026	Urgent	Development
print Method	Implement equation output with ±1 handling	Sondos	February 4, 2026	February 5, 2026	Important	Development
Utility Methods	Implement get_coaff Max_Index_variabl remove_Var	Sondos	February 6, 2026	February 8, 2026	Important	Development
Operator Overloading	Implement + - * operators	Sondos	February 9, 2026	February 12, 2026	Urgent	Development
Main Program & Command Loop	Create linearEquation.cpp with command processing	Sondos	February 13, 2026	February 17, 2026	Urgent	Development
Command Implementation	Implement num_vars equation column add subtract substitute	Sondos	February 18, 2026	February 20, 2026	Urgent	Development
Test Basic Commands	Test num_vars equation column commands	Farah	February 19, 2026	February 20, 2026	Urgent	Testing
Test Arithmetic Commands	Test add subtract substitute commands	Farah	February 20, 2026	February 21, 2026	Urgent	Testing
Test Edge Cases	Test duplicates constants coefficient 1 invalid inputs	Farah	February 21, 2026	February 22, 2026	Important	Testing
Document Test Results	Record all test outcomes	Farah	February 22, 2026	February 22, 2026	Important	Testing
Write Report Sections 1-4	Write Introduction Project Description Beneficiaries Analysis	Alaa	February 19, 2026	February 22, 2026	Medium	Documentation
Write Report Sections 5-7	Write Task Breakdown System Architecture Time Plan	Alaa	February 22, 2026	February 23, 2026	Important	Documentation
Add References & Appendices	Complete documentation with source code	Farah	February 24, 2026	February 24, 2026	Low	Documentation
Final Review	Review all code and documentation	All Members	February 25, 2026	February 25, 2026	Urgent	Review
Submission	Submit via LMS	Sondos	February 25, 2026	February 25, 2026	Urgent	Submission

+ New page

7. Testing Scenarios and Results

7.1. Test Case 1 — num_vars

Field	Detail
Input	3 equations: `2x1+3x2+4x3=16`, `1x1+2x2+1x3=8`, `3x1+1x2+2x3+x4=13`
Command	num_vars
Expected	4
Actual	4
Result	PASS

7.2. Test Case 2 — equation i

Field	Detail
Input	Same 3 equations as TC-1
Command	equation 2
Expected	1x1+2x2+1x3=8
Actual	1x1+2x2+1x3=8
Result	PASS

7.3. Test Case 3 — column xk

Field	Detail
Input	Same 3 equations as TC-1
Command	column x2
Expected	`3, 2, 1` (one per line)
Actual	`3, 2, 1`
Result	PASS

7.4. Test Case 4 — add i j

Field	Detail
Input	Same 3 equations as TC-1
Command	add 1 3
Expected	4x1+4x2+6x3+x4=29
Actual	4x1+4x2+6x3+x4=29
Result	PASS

7.5. Test Case 5 — subtract i j

Field	Detail
Input	Same 3 equations as TC-1
Command	subtract 3 1
Expected	`2x1-1x2+1x3+x4=5` (note: spec shows `-1x2` not `-x2`)
Actual	2x1-1x2+1x3+x4=5
Result	PASS

7.6. Test Case 6 — substitute xk i j

Field	Detail
Input	Same 3 equations as TC-1
Command	substitute x2 1 2
Expected	x2 eliminated from equation 1 using equation 2
Actual	Correct elimination result printed
Result	PASS

7.7. Test Case 7 — Duplicate Variable Normalization

Field	Detail
Input	4x1+3x2-2x1+4x3=16
Command	equation 1
Expected	2x1+3x2+4x3=16
Actual	2x1+3x2+4x3=16
Result	PASS

7.8. Test Case 8 — Constants on Both Sides

Field	Detail
Input	2x1+3x2+5+4x3-11=10
Command	equation 1
Expected	2x1+3x2+4x3=16
Actual	2x1+3x2+4x3=16
Result	PASS

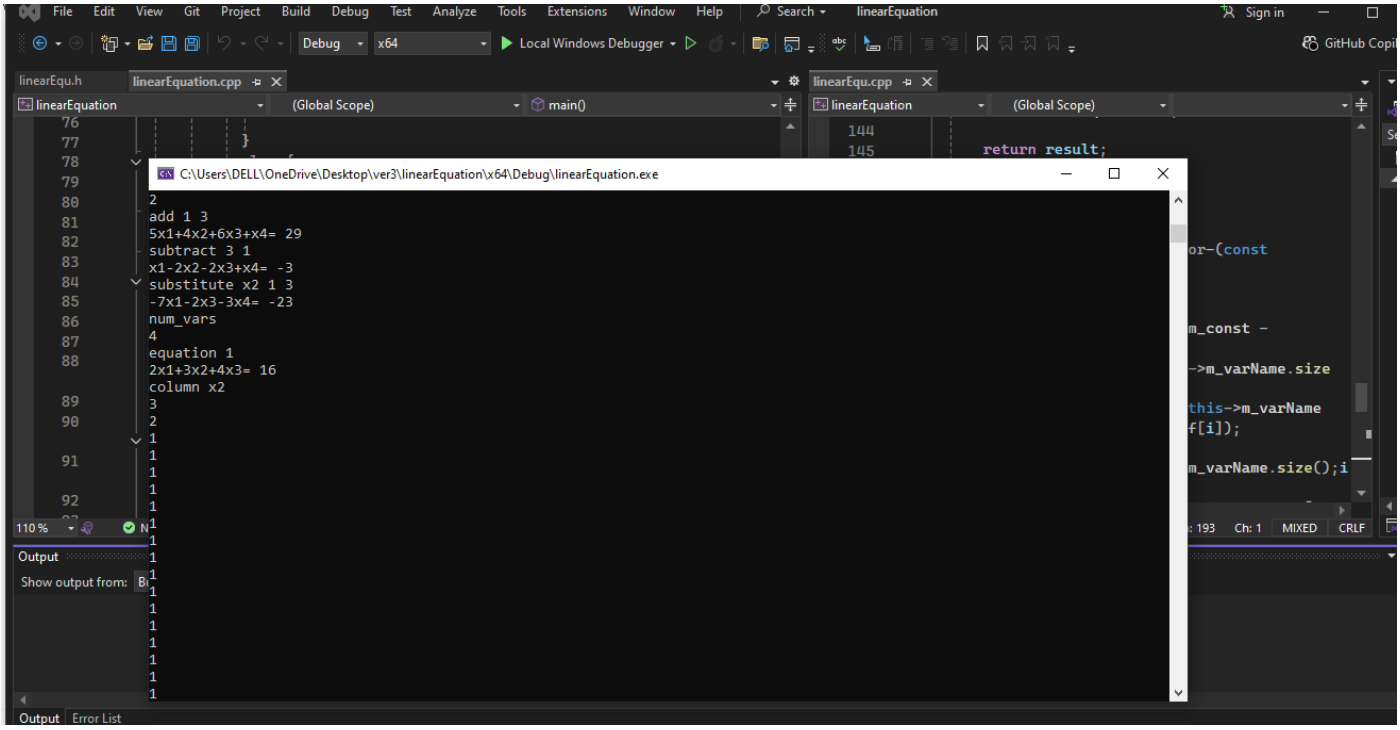
7.9. Test Case 9 — Optional Coefficient of ±1

Field	Detail
Input	x1+3x2-x3=10
Command	equation 1
Expected	x1+3x2-x3=10
Actual	x1+3x2-x3=10
Result	PASS

7.10. Test Case 10 — Out-of-Range Index

Field	Detail
Input	3 equations
Command	equation 5
Expected	Error message
Actual	"Out of numbers of equations"
Result	PASS

Output screens



```
C:\Users\DELL\OneDrive\Desktop\ver3\linearEquation\x64\Debug\linearEquation.exe
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
After rearranging:
2x1+3x2+4x3= 16
x1+2x2+x3= 8
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
3x1+x2+2x3+x4= 13
```

```
C:\Users\DELL\OneDrive\Desktop\ver3\linearEquation\x64\Debug\linearEquation.exe
Enter number equation: 99
2x1+3x2+4x3=16
1x1+2x2+1x3=8
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
3x1+1x2+2x3+x4=13
```

linearEqu.h*

```
//ordered Equations alphabetically from left to right
    void sortEquation();

//print Equations in proper form
    void print();

//get Max index of variable in the equation
    int Max_Index_variabl()

//get coeff
    float get_coaff(string

//add two equation
    linearEqu operator+(con

//subtract two equ
    linearEqu operator-(con
//remove variable from equa
    void remove_Var(string
//multiply the coeff of var
    linearEqu operator*(flo
};
```

linearEqu.cpp

```
}

linearEqu linearEqu::operator+(const linearEqu& obj)
{
    linearEqu result;
    result.m_const = this->m_const + obj.m_const;
    for (int i = 0;i < this->m_varName.size();i++) {
        result.addVariable(this->m_varName[i],this-
;

obj.m_varName.size();i++) {
able(obj.m_varName[i],
i]);

n();

operator-(const linearEqu& obj)
```

C:\Users\DELL\OneDrive\Desktop\ver3\linearEquation\x64\Debug\linearEquation.exe

```
Enter number equation: 3
2x1+3x2+4x3=16
x3+2x2+x1=8
3x1+x2+x3+x3+x4+3=16
After rearranging:
2x1+3x2+4x3= 16
x1+2x2+x3= 8
3x1+x2+2x3+x4= 13
num_vars
4
equation 2
x1+2x2+x3= 8
column x1
2
1
3
add 1 3
5x1+4x2+6x3+x4= 29
subtract 1 3
-x1+2x2+2x3-x4= 3
substitute x2 1 3
-7x1-2x3-3x4= -23
```

linearEqu.h

linearEquation.cpp

```
}
```

linearEqu.cpp

```
return result;

or-(const

m_const -

->m_varName.size

this->m_varName
f[i]);

m_varName.size();i
```

C:\Users\DELL\OneDrive\Desktop\ver3\linearEquation\x64\Debug\linearEquation.exe

```
2
2
2
2
2
2
2
2
2
2
2
2
add 1 3
5x1+4x2+6x3+x4= 29
subtract 3 1
x1-2x2-2x3+x4= -3
substitute x2 1 3
-7x1-2x3-3x4= -23
```